MINI PROJECT REPORT

RELIEFPLAN AI

AI Assisted Emergency Resource Allocation
and Decision Support System

CSI3001 CLOUD COMPUTING AND METHODOLOGIES

PALLI PAVAN KUMAR - 23MID0181

DUDDU SATYA SAI VISHNU VARDHAN - 23MID0163

DARISI RITHVIK - 23MID0209

G TARUNI - 23MID0170

School of Computer Science and Engineering SCOPE

Integrated M Tech
in
Computer Science and Engineering
With Specialization in Data Science

October 2026



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

https://github.com/palli-pavan/dia

1 ABSTRACT

ReliefPlan AI is a working decision support prototype that combines disaster observations, damage classification, resource demand estimation, constrained allocation and local relief fund bookkeeping. The system allows users to enter newly affected places with their own coordinates, population and observed damage. It therefore supports a changing scenario instead of limiting the demonstration to predefined locations.

The application uses a React dashboard and a FastAPI backend. A saved XGBoost model predicts damage severity, while a separate priority calculation ranks locations using impact, demand, shortage, vulnerable residents and available accessibility information. A linear programming solver allocates food, drinking water and medical kits within the available stock. SHAP feature contributions help explain the classifier output. Plans, dispatch confirmations, simulations, fund entries and audit events persist in SQLite.

The current demonstration contains Vijayawada, Guntur, Tenali, Mangalagiri, Amaravati and Machilipatnam. Its synthetic observations describe 26,700 residents, including 10,400 entered as needing assistance. With 15,000 food rations, 15,000 litres of water and 900 medical kits over 72 hours, the solver reports an optimal solution and 24.04 percent demand coverage for each resource. The shared donation ledger contains a clearly identified INR 7,00,000 demo receipt and the same available balance.

Verification on 30 October 2026 completed all 90 automated tests. The submitted project includes editable source, model artifacts, a prebuilt dashboard and reproducible launch commands. The model was trained and evaluated on synthetic records; the demonstrated results establish software behavior rather than field accuracy. Dispatch steps are local confirmations, and the donation module does not process payments.

Keywords

Disaster response, emergency resource allocation, XGBoost, linear programming, SHAP, FastAPI, React, SQLite and donation ledger.

Project deliverables

The repository provides the complete implementation. This report explains the architecture and implementation steps, presents the main code extracts, documents each dashboard page and summarizes the verified outputs.

Source repository: https://github.com/palli-pavan/dia

2 INTRODUCTION

Emergency planning requires several decisions to be made from the same set of observations. A coordinator needs to identify affected places, estimate the number of residents requiring support, determine available supplies and explain which requests remain unmet. When these activities are maintained in separate spreadsheets, changes to population or stock can produce inconsistent results across the plan.

We developed ReliefPlan AI to bring these activities into one reviewable workflow. The Describe page records a scenario, its affected locations and its available supplies. AI analysis presents predicted damage, relative priority, geographical markers and resource coverage. The Response plan converts allocations into local action records. What-if and audit supports controlled changes to stock or population. Donations and funds maintains the shared INR ledger independently of the numerical allocation process.

2 1 Scope of the implemented project

Users can add, edit and remove custom places, retain their drafts in the browser and reopen saved scenarios. Coordinates and observations entered for a new place flow through prediction, demand, allocation, explanation and export. An explicit affected population takes precedence over the model-derived estimate. The six-place Andhra flood demonstration is a convenience for review; each value remains editable.

2 2 Relevance to cloud computing methodologies

The project separates presentation, HTTP application services and persistent data. This separation supports a later move to a managed application server and database. In the submitted implementation, one local FastAPI process serves both the API and the built React files at port 8008. Cloud hosting, distributed deployment and a managed database are future deployment options; the report does not claim that they have been implemented.

2 3 Design approach

We prioritized a complete local demonstration with validated inputs, persistent decisions, readable error messages and visible assumptions. The application shows shortages directly and retains the original saved plan when a stress test is run. Human review remains necessary before acting on any recommendation.

3 LITERATURE REVIEW

The implementation draws on published methods for tree boosting, model interpretation and linear optimization. These sources support the component choices. They do not independently validate the application for disaster operations.

Work and authors	Contribution	Use in this project
XGBoost A Scalable Tree Boosting System T Chen and C Guestrin 2016 [1]	A scalable framework for gradient boosted tree models.	Damage severity classification from the supplied tabular observations.
A Unified Approach to Interpreting Model Predictions S Lundberg and S I Lee 2017 [2]	SHAP assigns additive contributions to features for a prediction.	Local explanation of the predicted damage class.
Parallelizing the dual revised simplex method Q Huangfu and J A J Hall 2018 [3]	High performance methods for solving linear programming problems.	Background for the HiGHS optimization tools used through SciPy.

3 1 Implications for the design

Prediction and allocation solve different problems. A classifier produces a damage category, whereas allocation must also respect the current stock and demand. Explanations of the classifier are therefore presented separately from the priority calculation and allocation rules. This distinction prevents a damage label from being presented as the complete reason for an allocation.

3 2 Gap addressed by the prototype

The practical integration problem addressed here is maintaining one consistent scenario across custom location entry, model output, resource planning, local action tracking and fund records. We use the same saved configuration for the baseline and stress tests, and we preserve an audit trail for decisions and ledger corrections. This is a software integration objective rather than a claim of a new learning or optimization algorithm.

4 OBJECTIVES

1. Accept newly affected places using their own coordinates, population, vulnerable group counts and damage observations.

2. Classify damage severity using the saved model and explain its output with feature contributions.

3. Estimate food, water and medical kit requirements for the selected planning period.

4. Allocate resources within stock limits while reserving a feasible common service fraction.

5. Compare allocation strategies and expose uncovered demand at each location.

6. Maintain saved scenarios, sequential dispatch confirmations and an auditable stress-test history.

7. Record received donations, pledges, expenses and corrections using exact INR amounts.

8. Provide a usable five-page dashboard and a reproducible source package for project review.

5 PROBLEM STATEMENT

Given a set of affected locations, a planning duration and a limited stock of food, water and medical kits, the system must produce a transparent allocation plan. Every allocation must remain non-negative, must not exceed the location demand and must respect the total stock available for that resource. Newly entered locations must be evaluated using their own observations.

The system must also make uncertainty and local operational status visible. Predicted damage must remain distinguishable from user-entered affected counts; dispatch confirmations must be traceable; pledged funds must remain separate from received money; and expenses must not exceed the available received balance. The result must be understandable through the dashboard and reproducible from saved inputs.

6 SYSTEM ARCHITECTURE

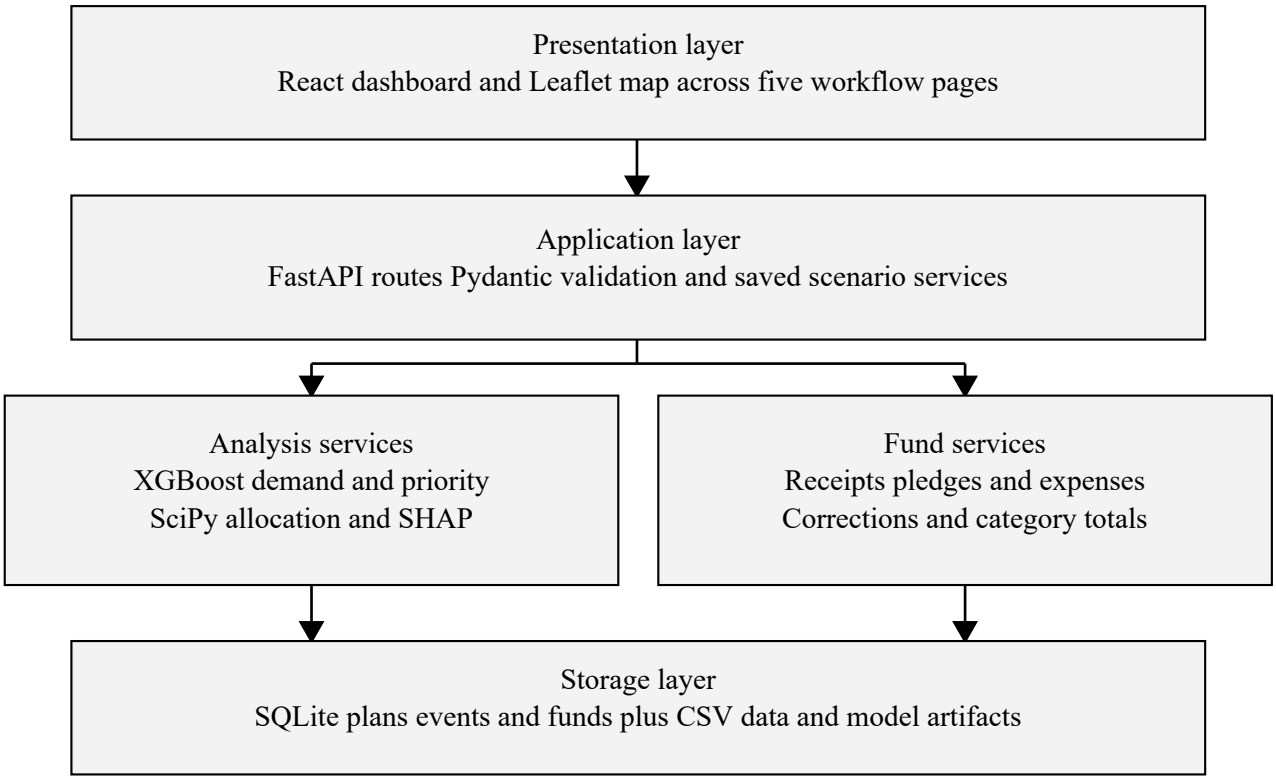


Figure 1 Overall system architecture

Figure 1 shows the implemented three-layer structure. The browser calls the FastAPI routes on the same origin. Analysis services transform the scenario into predictions and allocations; fund services validate and persist manual bookkeeping. Both use local persistence, but a donation does not automatically increase inventory or change an allocation.

6 1 Presentation layer

ReviewApp.jsx manages scenario state, navigation, saved plans and analysis results. LocationEditor.jsx provides the custom place dialog. FundsPanel.jsx manages ledger forms, summaries, targets and corrections. CSS defines the responsive desktop sidebar and compact navigation.

6 2 Application and storage layers

server.py exposes the planning API and serves frontend/dist. funds_api.py exposes fund routes. core/planning.py prepares consistent model inputs and stocks. core/store.py persists plans and events; core/funds.py adds ledger tables and transactions. The runtime entry point is run.py.

6 3 DATA FLOW DIAGRAM

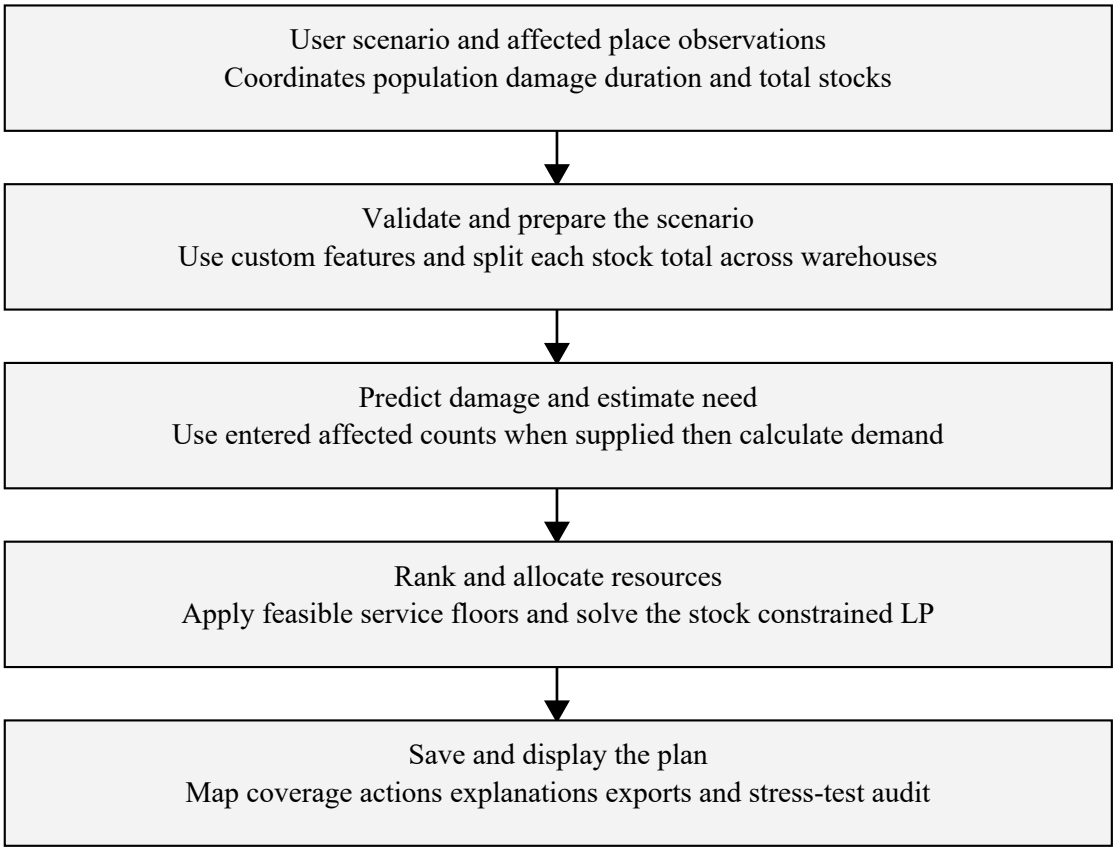


Figure 2 Planning data flow from observations to saved outputs

Figure 2 traces the numerical pipeline. Optional fund entries follow a separate flow: validate the amount and status, check the available balance, save the entry and audit event in one transaction, then refresh the summaries. This separation keeps donation bookkeeping from being confused with resource stock.

7 METHODOLOGY

We used a modular implementation process. We first defined the observations and constraints, then connected model inference, demand estimation, priority scoring and allocation through one shared scenario preparation function. We added persistent plans and action states, followed by custom locations, scenario comparison, explanations and the fund ledger. Browser checks and automated regressions verified the resulting workflow.

7 1 DEVELOPMENT AND IMPLEMENTATION STEPS

1 Define the input schema

Set limits for coordinates, population, damage counts, duration, stock and the common service fraction. Reject inconsistent vulnerable group or affected population counts.

2 Prepare local data and artifacts

Load the synthetic CSV study records and the saved model, label encoder, SHAP explainer and evaluation metrics. Use custom observations directly for newly entered places.

3 Build the planning pipeline

Calculate predicted damage, affected population, daily resource demand, relative priority and a stock-constrained allocation using the same configuration.

4 Add persistent decisions

Store each successful plan and record sequential action changes, simulations and local bulletins. Keep baseline plans available for later review.

5 Implement the five dashboard pages

Connect scenario entry, map and ranking, response filters, stress tests and the donation ledger. Add loading, validation and empty states.

6 Verify and package

Run the 90-test suite, build and lint the frontend, inspect the working pages, and publish source with the model artifacts and built dashboard.

7 2 Technologies used

Technology	Role
Python and FastAPI	Validated API and local application server
React Vite and Leaflet	Dashboard build interactive forms and map
XGBoost and scikit learn	Classification and recorded model evaluation
SciPy HiGHS and SHAP	Constrained allocation and prediction explanations
SQLite Pandas and NumPy	Persistent records and numerical data processing

8 IMPLEMENTATION

8 1 Detailed working mechanism

For each custom place, the system combines the minor, major and destroyed building counts to obtain total reported building damage. The six model features are building damage, minor damage, major damage, destroyed buildings, population density and hazard magnitude. The model returns a damage class and class probabilities. These are not a direct count of people needing assistance.

If an affected population is entered, that count is used. Otherwise, the application derives an estimate from the predicted severity and planning assumptions. Demand is then computed from the affected population, per-person daily rate and duration in days. In the flood demonstration the rates are 2 food rations, 2 litres of water and 0.12 medical kits per person per day.

8 2 Demand and service constraints

```
demand[i, k] = affected_population[i] * daily_rate[k] * duration_days
floor[k] = min(requested_floor, supply[k] / total_demand[k])
floor[k] * demand[i, k] <= allocation[i, k] <= demand[i, k]
sum(allocation[:, k]) <= supply[k]
```

The requested service fraction is 25 percent in the demo. Because the available stock can support only about 24.04 percent of each resource demand, the feasible floor is reduced to that shared percentage. Every area then receives a feasible share. Under tighter scarcity this can leave no surplus for priority differentiation; identical coverage is therefore an expected outcome, not evidence that the ranking was ignored.

8 3 Priority and explanation

The default priority weights are 0.25 for impact, 0.25 for demand, 0.20 for shortage, 0.15 for medical need and 0.15 for accessibility penalty. Components are normalized within the scenario. New custom places have no surveyed road connections in the supplied dataset, so the application does not infer road closures, routes or arrival times for them. SHAP describes the damage classifier behavior separately; it does not establish causation.

8 4 MAIN CODE FOR INPUT VALIDATION

The following extracts are from the submitted source. They illustrate the main implementation decisions; the repository contains the complete functions and surrounding imports. Code is shown without line numbers.

Custom affected population validation

Source: server.py AffectedLocation

```
@model_validator(mode="after")
def coherent_population(self):
    if (self.affected_population is not None
            and self.affected_population > self.population):
        raise ValueError(
            "Affected population cannot exceed resident population.")
    if self.children_population + self.elderly_population > self.population:
        raise ValueError(
            "Children and elderly counts combined cannot exceed "
            "resident population.")
    return self
```

This check prevents the affected count from exceeding the resident population and rejects overlapping totals where children plus elderly residents exceed the population.

Shared analysis and simulation preparation

Source: core/planning.py execute_scenario

```
def execute_scenario(config, water_change=0.0, population_change=0.0):
    data, rates = prepare_scenario(config)
    result = run_full_pipeline(
        datasets=data,
        model_path=str(ROOT / "models/best_model.pkl"),
        encoder_path=str(ROOT / "models/label_encoder.pkl"),
        duration_days=config["duration_hours"] / 24,
        per_capita_needs=rates,
        affected_population_multiplier=SEVERITY_MULTIPLIERS[config["severity"]],
        minimum_service_fraction=config["minimum_service_fraction"],
        resource_supply_deltas={"Water": water_change},
        population_delta_pct=population_change)
    result["parameters"].update(config)
    result["planning_rates"] = rates
    return result
```

Both baseline analysis and stress tests call this function. Custom observations, duration, resource rates, planning severity and service fractions are therefore handled consistently. The saved baseline result remains available after a simulation.

8 5 MAIN CODE FOR ALLOCATION AND FUNDS

Feasible service floor and linear programming

Source: core/optimizer.py optimize_resource_allocation

```
floor = min(minimum_service_fraction, tot_supply / demands.sum())
service_floors[r_type] = float(floor)
bounds = [(float(floor * d), float(d)) for d in demands]

# Solve with SciPy Highs LP solver
res = linprog(c=c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method="highs")
```

The LP bounds reserve a feasible share for each location, cap allocations at demand and limit the sum to available supply. The objective coefficients are based on priority divided by demand. Full precision is retained internally so display rounding cannot overdraw stock.

Exact money conversion at the API boundary

Source: funds_api.py record_entry

```
record = req.model_dump(exclude={"request_id", "amount"})
record.update(
    amount_paise=int(req.amount * 100),
    scenario_name=scenario_name, area_name=area_name)
try:
    entry = funds.create_entry(req.request_id.hex, record)
except ValueError as exc:
    raise HTTPException(409, str(exc)) from exc
```

Expense balance guard inside a transaction

Source: core/funds.py create_entry

```
with connection() as db:
    db.execute("BEGIN IMMEDIATE")
    # Duplicate request handling precedes this guard.
    if record["kind"] == "EXPENSE":
        available = _totals(db)["available_paise"]
        if record["amount_paise"] > available:
            raise ValueError(
                "Expense exceeds available received funds. "
                "Pledges cannot be spent.")
```

Amounts are validated as Decimal and stored as integer paise. The transaction and unique request ID prevent concurrent overspending and duplicate submissions. Corrections void the entry while retaining its history. Extracts have been formatted for the page; the last example abbreviates surrounding duplicate-check code.

8 6 DESCRIBE DASHBOARD

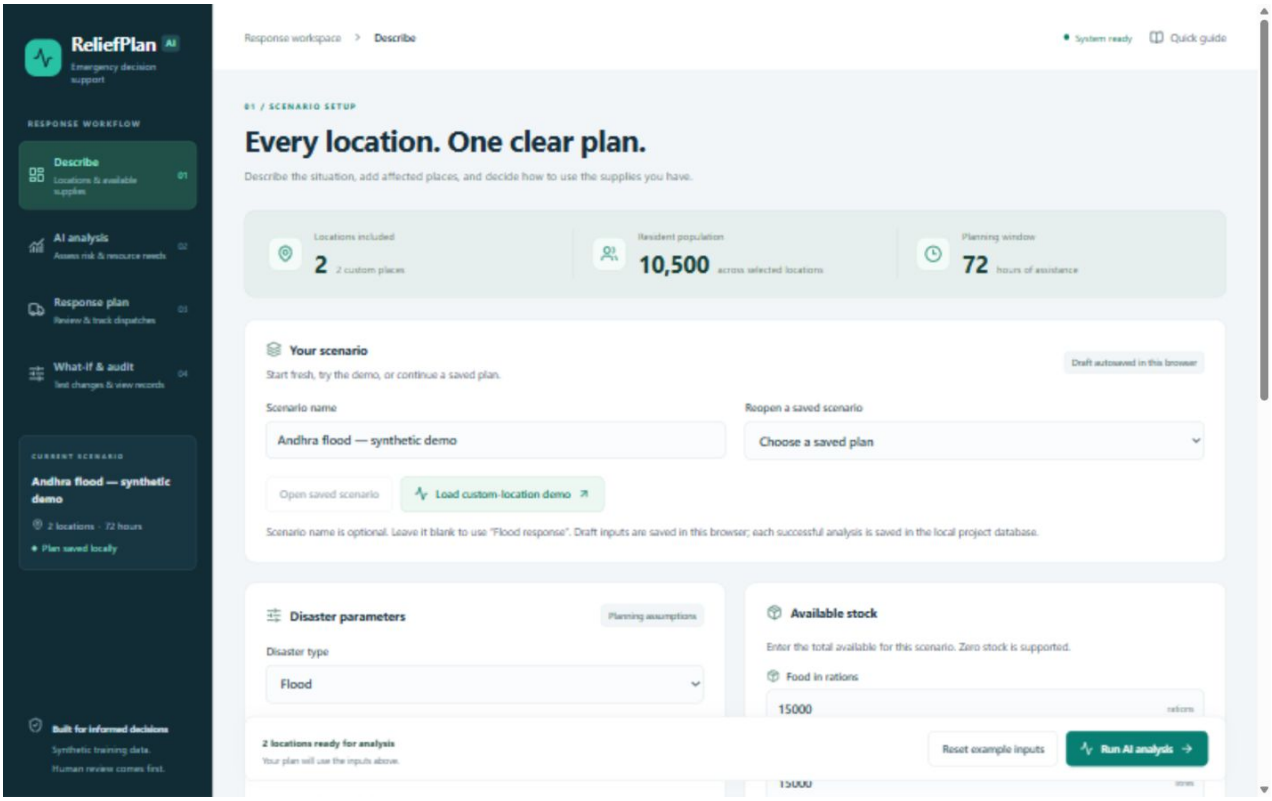


Figure 3 Describe page showing scenario inputs stock and custom location controls

Figure 3 shows the desktop entry page. The user enters a scenario name, disaster type, planning severity, response duration, service fraction and total stock. Example areas can be removed so that the calculation uses only custom places. A blank scenario name receives a readable default, and unfinished inputs are saved in the browser.

The desktop image records the earlier two-place demonstration layout. The current demo has been expanded to six places in the same page; the location values used in the verified result are listed in Table 1 on the following page. The Add sample locations action appends missing sample places while preserving the user inputs.

Interaction sequence

Open Describe, load the custom-location demo, inspect the values and select Run AI analysis. To enter a different affected place, select Add location and provide its own observations. A successful analysis saves a new plan and opens the assessment page. Saved plans remain available from the scenario selector.

8 7 CUSTOM LOCATION ENTRY AND DEMONSTRATION DATA

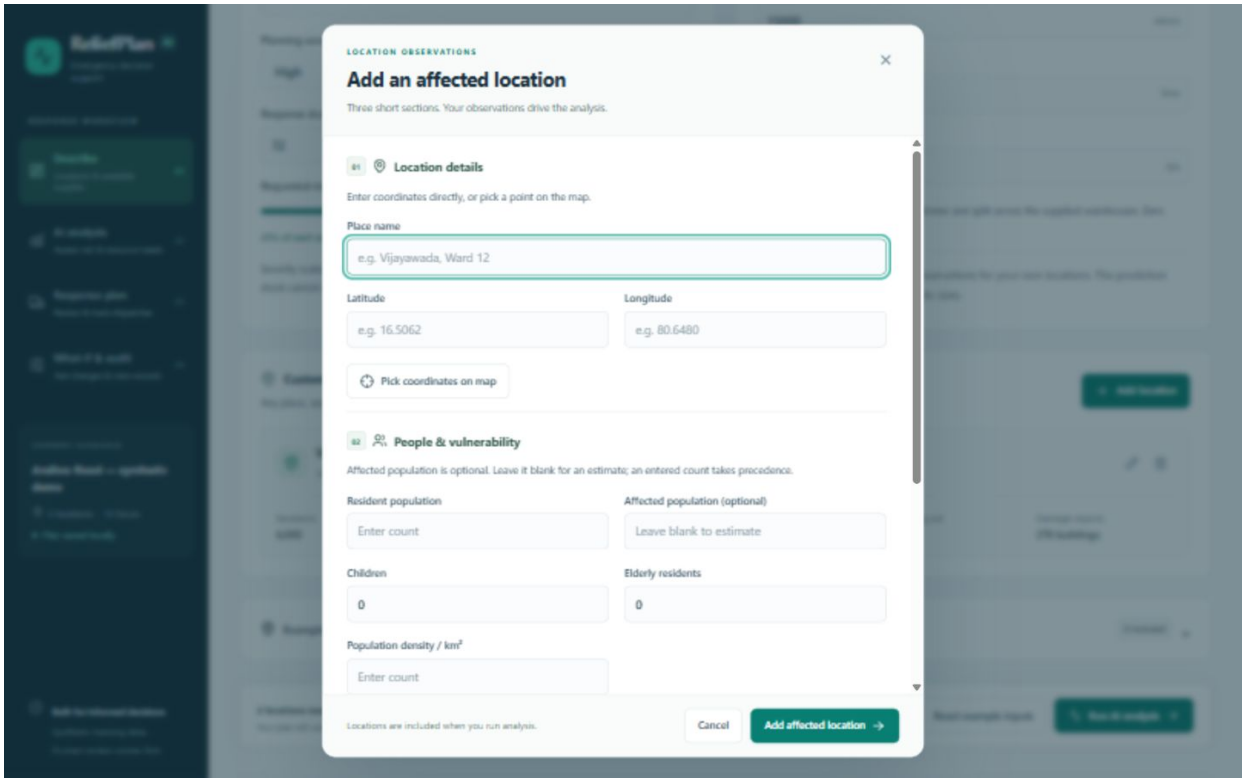


Figure 4 Custom location dialog with location people and observed damage fields

Figure 4 shows the focused input dialog. Place details, people and vulnerability, and observed damage are grouped into separate sections. Coordinates can be entered directly or picked on the map. Validation prevents missing names, invalid coordinates and inconsistent population counts. All counts below are synthetic observations for the review demonstration.

Table 1 Six affected locations used in the verified flood scenario

Location	Residents	Needing aid	Damage reports
Vijayawada	6,000	2,400	530
Guntur	4,500	1,500	270
Tenali	3,800	1,200	230
Mangalagiri	4,200	1,800	345
Amaravati	3,200	1,400	255
Machilipatnam	5,000	2,100	440

8 8 AI ANALYSIS DASHBOARD

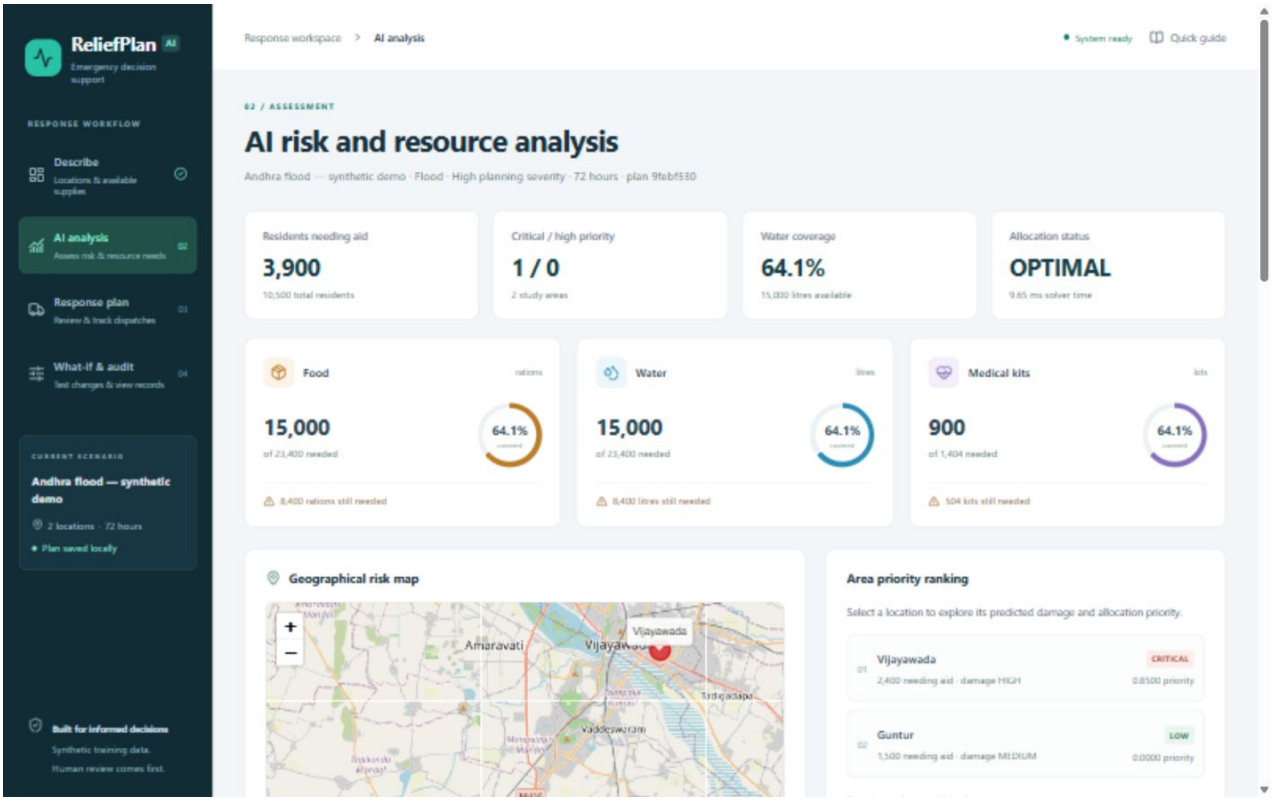


Figure 5 AI analysis page with resource coverage map markers and area priority ranking

Figure 5 presents the assessment layout. Resource cards summarize allocated quantities, demand and remaining need. The Leaflet map uses the actual coordinates supplied for each place. A marker or ranking row selects an area for inspection. The explanation section shows the predicted class, model confidence and feature contributions.

The ranking is relative to the selected scenario. Predicted damage severity and allocation priority are separate outputs: an area may have a high predicted damage class but a different relative priority after demand and vulnerable residents are considered. Input warnings identify observations outside the synthetic training range.

Verified six-location output

The current run reports 26,700 residents, 10,400 residents needing assistance, one critical-priority location and one high-priority location. Food and water demand are each 62,400 units over 72 hours; medical demand is 3,744 kits. Supplies cover 24.04 percent of each demand. The solver status is OPTIMAL. The desktop figure illustrates the earlier two-place scenario, while these values are from the current saved six-place run.

The resource table and exported plan expose demand, available stock, allocation, shortage, coverage and service floor. Different strategies may redistribute the same stock while leaving the same total shortage; the interface therefore also compares minimum and priority-weighted coverage.

8 9 RESPONSE PLAN DASHBOARD

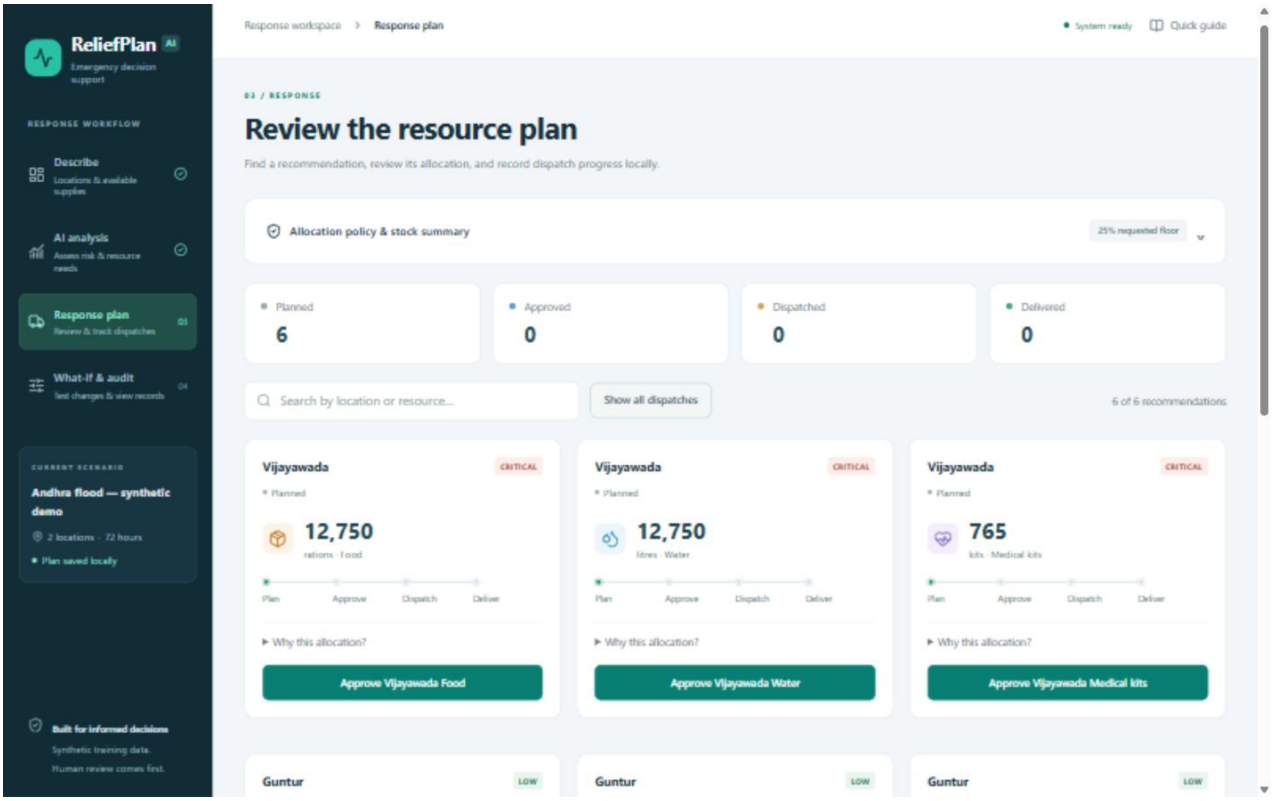


Figure 6 Response plan page with searchable allocation actions and local dispatch stages

Figure 6 shows the action review page. Cards identify the location, resource and recommended allocation. The user can search by place or resource and filter by the current dispatch stage. Stage counters summarize the local workflow.

Sequential local action states

The supported progression is Planned to Approved to Dispatched to Delivered. Updates are saved as audit events and are restored when the plan is reopened. The backend validates the requested action and stage transition so an unrelated plan or skipped stage cannot silently change the record.

These stages are confirmations entered into the local project. They do not contact vehicles, reserve warehouse inventory or provide an arrival time. An allocation remains a planning quantity and may contain fractions because the implemented solver is continuous rather than an integer packaging model.

Exports and traceability

The plan CSV contains location-level outputs. The audit CSV allows a reviewer to inspect the associated decisions. Reopening an older scenario restores its saved inputs and result instead of replacing it with the latest browser draft. The screen shown here is the desktop layout from the earlier two-place demonstration; the same controls operate on the six-place plan.

8 10 WHAT IF AND AUDIT DASHBOARD

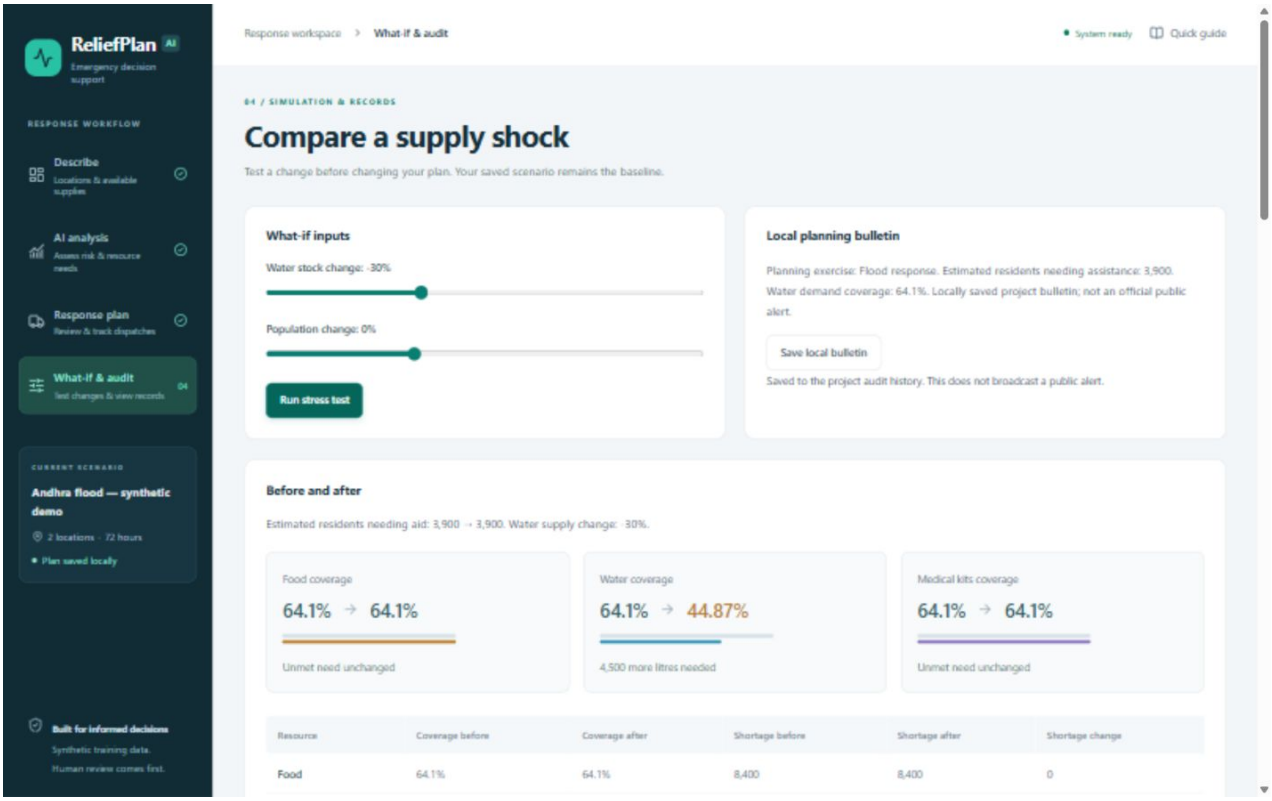


Figure 7 What if and audit page displaying stock changes comparison bars and recorded events

Figure 7 shows the stress-test workflow. The user changes the water-stock percentage or population percentage and runs the test against a selected saved plan. The result compares resource demand, allocation, shortage and coverage before and after the change. The baseline saved plan remains unchanged.

Demonstration of a water shortage

For the six-location scenario, a 30 percent reduction changes water supply from 15,000 litres to 10,500 litres. With the same demand of 62,400 litres, water coverage falls from 24.04 percent to approximately 16.83 percent and the uncovered requirement becomes 51,900 litres. Food and medical stock remain at their scenario values. These arithmetic values describe the entered demonstration assumptions.

Audit and local bulletin

The audit panel records the simulation parameters and local decisions. The optional bulletin is explicitly a planning exercise message saved in the project; it is not an official public alert and does not send an external notification. This figure illustrates the desktop comparison layout from the earlier demonstration.

8 11 DONATIONS AND FUNDS DASHBOARD

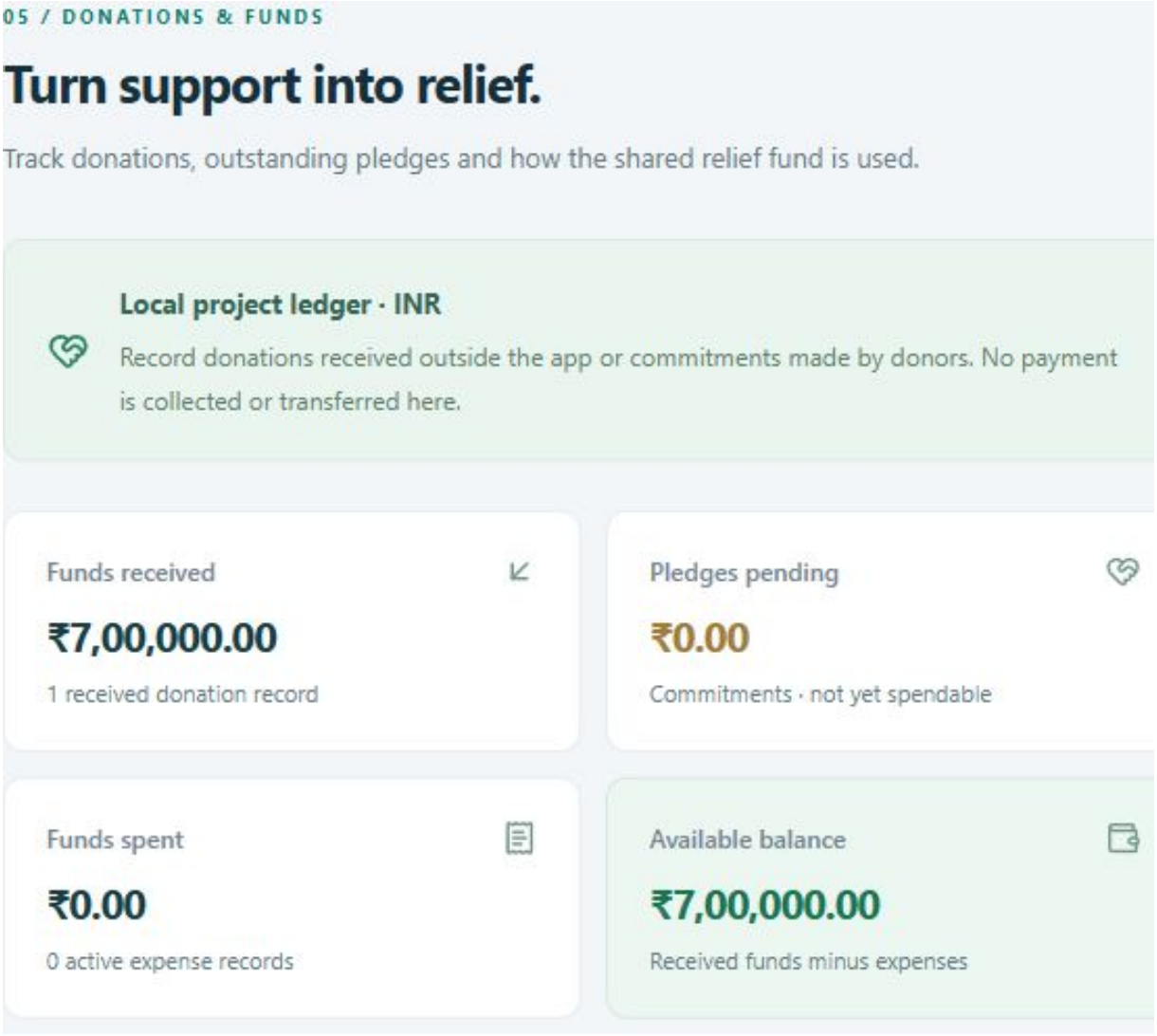


Figure 8 Current donation summary with the INR 7 lakh project review demo receipt

Figure 8 shows the current ledger summary captured for this report. Funds received and available balance are both INR 7,00,000. Pending pledges and expenses are zero. The entry is labeled Project review demo donation, uses reference DEMO-REVIEW-700000 and states that no actual payment was collected or transferred.

The page records donations, pledges and categorized expenses, optionally linked to a saved scenario and affected area. Pledges do not become spendable until marked received. Expenses cannot exceed the received balance. A fundraising target tracks progress separately. Search, filters and CSV export support review; incorrect records can be voided with a reason while their history remains visible.

The shared ledger is persistent on this computer. A fresh source installation starts with an empty database; the demo receipt is a local sample record rather than a real financial transaction. The application does not collect online payments, issue tax receipts or convert donations into resource inventory.

8 12 RESULTS AND VALIDATION

The current saved plan was generated on 30 October 2026 using the six-place flood scenario. Solver runtime in this run was 15.7 ms; this is one observed optimization time and is not an end-to-end performance benchmark.

Resource	Demand	Allocated	Unmet need	Coverage
Food	62,400	15,000	47,400	24.04%
Water	62,400	15,000	47,400	24.04%
Medical kits	3,744	900	2,844	24.04%

Recorded model evaluation

The supplied evaluation uses 800 synthetic samples, split into 640 training and 160 test records. Candidate models were compared using macro F1. These saved metrics are not a new training run or an estimate of real disaster accuracy.

Candidate model	Accuracy	Macro F1
Logistic Regression Baseline	67.50%	0.6765
Random Forest	78.75%	0.7183
XGBoost	81.87%	0.7366

Software verification

All 90 automated tests passed in 23.06 seconds on 30 October 2026. Coverage includes supply limits, common service floors, zero-stock behavior, custom coordinates and observations, affected-count overrides, scenario persistence, dispatch transitions, simulation consistency, explanations, fund precision, duplicate submission handling, corrections and concurrent expense limits. The production frontend build and lint check also passed.

Observed limitations

The examples and training records are synthetic. Custom road connectivity is unavailable, allocations are divisible, and the app is a local single-process prototype. A valid solver result confirms the mathematical constraints for the entered inputs; it does not establish that those inputs are complete or accurate.

9 CHALLENGES AND SOLUTIONS

Challenge	Implementation response
Only default locations could be studied	Added validated custom places with their own observations and coordinates throughout the pipeline.
Empty scenario names produced validation errors	Added frontend and backend defaults with field-specific messages.
Stock could be duplicated across warehouses	Treat the entered value as one total and split it across the supplied warehouse records.
A requested service floor could be infeasible	Reduce the common floor to the largest stock-supported fraction before allocation.
Display rounding could violate stock limits	Retain full precision during optimization and round the displayed values.
A browser draft could retain the old two-place demo	Upgrade the original demo once and provide an append-only sample location button.
Pledges could be confused with spendable donations	Keep received and pledged amounts separate and validate expenses against received funds.
Retries or simultaneous expenses could duplicate or overdraw funds	Use unique request IDs and an immediate SQLite transaction for guarded writes.

9 1 Usability improvements

The dashboard uses grouped forms, readable validation messages, editable location cards, undo for a removal, search and stage filters. Responsive navigation keeps all five sections accessible on narrow screens. Empty ledger and loading states explain what the user can do next. Existing scenario inputs remain visible when missing sample locations are appended.

10 FUTURE ENHANCEMENTS

The next substantial improvement would be independent validation on suitable real disaster observations. A larger dataset, documented labels and separate event-level evaluation would be needed before using prediction quality as evidence for field operations. The current results are limited to synthetic training and demonstration records.

Operational extensions could add surveyed road networks, verified warehouse locations, inventory reservations, delivery capacity and integer packaging constraints. These additions should be introduced with dedicated data and tests rather than inferred from the current coordinate map.

For a hosted deployment, the static frontend and API could be packaged for a cloud application service or virtual machine, with persistent storage, backups and deployment monitoring. A managed relational database could replace SQLite for multi-user operations. These remain proposed enhancements, and no hosted production service is claimed in the submission.

Fund enhancements could add supporting vouchers, reconciliation exports and a distinct training dataset mode. Any future payment integration would require a separate payment service; the existing manual ledger should remain clearly distinguishable from actual collections.

11 REAL WORLD APPLICATIONS

The current prototype can support classroom exercises, NGO planning demonstrations, local relief allocation workshops and scenario comparisons. A coordinator can enter a new village or ward, compare resource needs with the available stock, inspect shortages and record simulated dispatch progress in one workspace.

The fund ledger illustrates how received support, outstanding pledges and expenditure can be kept distinct and reviewed alongside a scenario. These applications are educational or planning uses. Deployment in real emergency operations would require validated data, an operational review process and stronger integration with the responsible organizations.

12 CONCLUSION

ReliefPlan AI implements a complete local workflow from disaster observations to saved allocation plans and reviewable fund records. It accepts custom affected places, estimates resource needs, solves stock-constrained allocations, explains model predictions, tracks local action states and compares stress-test outcomes. The five dashboard pages make these stages accessible through a consistent interface.

The six-location demonstration produces an optimal allocation within the entered stock limits and makes the remaining shortage explicit. The INR 7 lakh demo donation shows the persistent bookkeeping workflow without implying a real payment. The passing 90-test suite and successful frontend checks provide reproducible evidence of the implemented software behavior.

The main achievement is the integration of prediction, optimization, persistence and review into one working project. The current limits are also explicit: synthetic data, a local runtime, incomplete operational road information and continuous allocation quantities. These boundaries guide the future work rather than being hidden behind the interface.

13 REFERENCES

[1] T Chen and C Guestrin. XGBoost A Scalable Tree Boosting System. KDD 2016.
https://arxiv.org/abs/1603.02754

[2] S Lundberg and S I Lee. A Unified Approach to Interpreting Model Predictions. NeurIPS 2017.
https://arxiv.org/abs/1705.07874

[3] Q Huangfu and J A J Hall. Parallelizing the dual revised simplex method. Mathematical Programming Computation 10 119-142 2018.
https://doi.org/10.1007/s12532-017-0130-5

[4] Project source and submission artifacts. ReliefPlan AI. Accessed 30 October 2026.
https://github.com/palli-pavan/dia

[5] Saved evaluation record. models/metrics.json in the submitted source.
View saved evaluation at the submitted source revision

APPENDIX SOURCE CODE AND EXECUTION

A Source repository

https://github.com/palli-pavan/dia

The implementation described in this report was pushed to the main branch. Source revision: d815fc491a1292d53ba88f6aa957e0fcd057c555. The report is stored separately under reports so it can be downloaded with the code.

Download source from GitHub: Download the main branch source ZIP

B Important source files

File	Purpose
run.py server.py and funds_api.py	Start the app and expose planning and fund routes
core/planning.py	Prepare consistent scenarios for analysis and simulation
core/model.py and core/explain.py	Classifiers and SHAP explanations
core/priority.py and core/optimizer.py	Relative priority and constrained allocations
core/store.py and core/funds.py	Persistent plans decisions and exact-money ledger
frontend/src/ReviewApp.jsx LocationEditor.jsx and FundsPanel.jsx	Five-page dashboard custom places and donations

C Run on the existing computer

```
cd D:\Disaster
.\START_PROJECT.cmd
```

The dashboard opens at http://127.0.0.1:8008/. Keep the terminal open while using the project. Press Ctrl+C to stop the server.

D Install and verify after extracting the source

```
cd emergency-resource-allocation
python -m pip install -r requirements-tested.txt
python run.py

python -m pytest tests -q
```

Python 3.13 is verified. The prebuilt frontend is included. Rebuilding requires Node.js; run npm ci and npm run build in frontend. A new installation creates an empty database. Add the synthetic donation through Donations and funds if needed for the demonstration.